

《AI大模型Ragent项目》——SSE流式解析与异步执行

来自： 拿个offer-开源&项目实战



马丁

2026年04月13日 09:34



上一篇拆解了 doChat 同步调用的完整流程——校验供应商、构建请求体、发送 HTTP POST、解析 JSON 响应、提取 choices[0].message.content，九个步骤一路走下来，很直接。

但在文章最后提到，AbstractOpenAIStyleChatClient 还有一个 doStreamChat 模板方法。同步调用的逻辑很好理解：发一个 HTTP POST，等响应回来，解析 JSON，返回 String。流式调用就不一样了——响应不是一次到位的，而是一行一行通过 SSE 协议推送过来的。读取过程是阻塞的（readUtf8Line() 会阻塞等待下一行数据到达），不能在 Tomcat 请求线程上跑，得提交到专用线程池异步执行。而且用户随时可能点停止生成，得有一套取消机制能中断正在进行的流式读取。

这一篇打开流式调用的黑盒，看看 doStreamChat 内部是怎么把 SSE 解析、异步执行、回调推送、取消控制串在一起的。

doStreamChat vs doChat：同步与流式的分叉点

1. doStreamChat 模板方法

先看 doStreamChat 的完整代码：

```
protected StreamCancellationHandle doStreamChat(ChatRequest request, StreamCallback callback, Mode
    AIModelProperties.ProviderConfig provider = HttpResponseHelper.requireProvider(target, provide
    if (requiresApiKey()) {
        HttpResponseHelper.requireApiKey(provider, provider());
    }

    JsonObject reqBody = buildRequestBody(request, target, true);
    Request streamRequest = newAuthorizedRequest(provider, target)
        .post(RequestBody.create(reqBody.toString(), HttpMediaTypes.JSON))
        .addHeader("Accept", "text/event-stream")
        .build();

    Call call = httpClient.newCall(streamRequest);
    boolean reasoningEnabled = isReasoningEnabledForStream(request);
    return StreamAsyncExecutor.submit(
        modelStreamExecutor,
        call,
        callback,
        cancelled -> doStream(call, callback, cancelled, reasoningEnabled)
    );
}
```

和上一篇的 doChat 放在一起对比，前半部分几乎一模一样：校验供应商配置 → 校验 API Key → buildRequestBody 构建请求体 → newAuthorizedRequest 构建 HTTP 请求。但从构建请求体开始，三个关键差异出现了：

维度	doChat（同步）	doStreamChat（流式）
请求体	buildRequestBody(request, target, false)	buildRequestBody(request, target, true), JSON 中多了 "stream": true
Accept 头	无特殊设置	.addHeader("Accept", "text/event-stream")
HTTP 执行	当前线程同步 httpClient.newCall(request).execute()	创建 Call 后交给 StreamAsyncExecutor.submit() 异步执行
响应解析	HttpResponseHelper.parseJson() 一次性 解析完整 JSON	OpenAIStyleSseParser.parseLine() 逐行 解析 SSE
结果返回	直接 return String	通过 StreamCallback 回调推送
取消控制	无（同步阻塞等待）	返回 StreamCancellationHandle，调用方可随时取消

错误处理	抛 <code>ModelClientException</code> ，由 <code>executeWithFallback</code> 捕获	<code>callback.onError()</code> 通知调用方
------	--	---------------------------------------

分叉点就在 `httpClient.newCall(streamRequest)` 这一行——同步调用直接 `.execute()` 等结果，流式调用把 `Call` 对象交给异步执行器，自己立即返回一个取消句柄。

2. 为什么流式读取必须异步

这个问题值得展开说说。
OkHttp 的流式响应通过 `BufferedSource.readUtf8Line()` 逐行读取。这个方法是阻塞调用——如果服务端还没推送新数据，`readUtf8Line()` 会一直等着，直到新的一行到达或者连接关闭。
大模型生成一个完整回答可能需要几十秒甚至几分钟（生长文本时），在这段时间里，线程一直被占着。如果把流式读取放在 Tomcat 的请求处理线程上，一个流式请求就占住一个 Tomcat 线程。Tomcat 线程池默认 200 个线程，假设同时有 50 个用户在流式对话，就有 50 个线程被占住。剩下的线程要处理其他所有 HTTP 请求——页面渲染、API 调用、健康检查等等。并发一上来，线程池就满了，新请求排队等待，整个系统响应变慢。
所以必须把流式读取提交到专用线程池 `modelStreamExecutor` 上异步执行。Tomcat 请求线程只负责构建 HTTP 请求和创建 `Call` 对象，然后立即返回取消句柄，不等待流式传输完成。真正的阻塞读取在专用线程池上进行，和 Tomcat 线程池互不干扰。

StreamCallback：回调体系

1. 接口定义

```
public interface StreamCallback {

    /**
     * 接收一次增量内容（Delta Token 或部分片段）
     */
    void onContent(String content);

    /**
     * 接收思考过程增量内容（如果模型支持）
     * 默认空实现，未支持思考的场景可以忽略
     */
    default void onThinking(String content) {
    }

    /**
     * 整个推理流程结束（全部内容推送完毕）
     */
    void onComplete();

    /**
     * 流式推送过程中出现异常
     */
    void onError(Throwable error);
}
```

四个方法分工明确：

- `onContent(String content)`——增量内容回调。模型每生成一段内容（可能是一个字、一个词、一句话，取决于模型的分片策略），就通过这个方法推一次。调用次数不定，由模型输出节奏决定
- `onThinking(String content)`——思考过程回调。深度思考模型（比如开启了 `enable_thinking` 的 Qwen）会在生成正式回答之前输出推理过程，通过这个方法推送。注意它是 `default` 空实现
- `onComplete()`——流结束通知。所有内容推送完毕后调用一次，通知上层做收尾动作（停止 loading 动画、滚动到底部等）
- `onError(Throwable error)`——异常通知。网络中断、解析异常、供应商报错等场景触发

2. 调用时序

正常流程的回调顺序：

```
onThinking("用户在问候")           ← 可选，深度思考模型才有
onThinking(", 我应该友好回应")       ← 可选
onContent("你")
onContent("好")
onContent("! ")
onComplete()                         ← 正好一次
```

异常流程：

```
onContent("你")
onContent("好")
onError(IOException)                 ← 任意时刻，一次
```

onComplete() 和 onError() 是互斥的终态——正常结束走 onComplete()，异常结束走 onError()，不会两个都调用。

2.1 onThinking 的默认空实现

onThinking 被声明为 default 方法，方法体为空。这个设计的意图是：大多数场景不关心模型的思考过程，只关心最终生成的内容。如果 onThinking 是一个抽象方法，所有 StreamCallback 的实现类都得写一个空的 onThinking 方法，哪怕根本用不到。default 空实现让不需要思考内容的调用方零成本忽略它——只实现 onContent、onComplete、onError 三个方法就够了。

3. 和同步调用返回值的对比

同步调用 doChat 的返回方式是直接 return String——调用方同步阻塞等待整个回答生成完毕，然后一次性拿到完整文本。
流式调用通过 StreamCallback 异步推送——调用方不阻塞等待，增量内容逐步到达。这种模式让前端可以实现逐字输出的打字机效果：收到一个 onContent("你") 就在页面上渲染一个字，不用等整个回答生成完才显示。用户体验上，从发问到看到第一个字的延迟（首字延迟，Time to First Token）大幅缩短。

OpenAIStyleSseParser：SSE 行解析

1. 为什么需要专门的解析器

大模型的流式响应走 SSE（Server-Sent Events）协议，但实际上用的是一个简化子集——只用了 data: 字段，没有 event:、id:、retry: 等标准 SSE 字段。而且 data: 后面的 payload 是 JSON，和非流式响应的结构有区别：

```
非流式响应：内容在 choices[0].message.content
流式响应：增量内容在 choices[0].delta.content
流结束标记 [DONE] 是 OpenAI 协议特有的约定，不在 SSE 标准里
```

所以需要专门的解析器来处理这些 OpenAI 协议特有的约定。OpenAIStyleSseParser 就是这个角色——它不是一个通用的 SSE 协议解析器，而是一个针对 OpenAI 兼容接口优化的 data-only SSE 消费者。

2. parseLine 完整代码

```
@NoArgsConstructor(access = lombok.AccessLevel.PRIVATE)
public final class OpenAIStyleSseParser {

    private static final String DATA_PREFIX = "data: ";
    private static final String DONE_MARKER = "[DONE]";

    static ParsedEvent parseLine(String line, Gson gson, boolean reasoningEnabled) {
        if (line == null || line.isBlank()) {
            return ParsedEvent.empty();
        }

        String payload = line.trim();
        if (payload.startsWith(DATA_PREFIX)) {
            payload = payload.substring(DATA_PREFIX.length()).trim();
        }
        if (DONE_MARKER.equalsIgnoreCase(payload)) {
            return ParsedEvent.done();
        }
    }
}
```

```
}

JsonObject obj = gson.fromJson(payload, JsonObject.class);
JsonArray choices = obj.getAsJsonArray("choices");
if (choices == null || choices.isEmpty()) {
    return ParsedEvent.empty();
}

JsonObject choice0 = choices.get(0).getAsJsonObject();
String content = extractText(choice0, "content");
String reasoning = reasoningEnabled ? extractText(choice0, "reasoning_content") : null;
boolean completed = hasFinishReason(choice0);

return new ParsedEvent(content, reasoning, completed);
}

private static boolean hasFinishReason(JsonObject choice) {
    if (choice == null || !choice.has("finish_reason")) {
        return false;
    }
    JsonElement finishReason = choice.get("finish_reason");
    return finishReason != null && !finishReason.isJsonNull();
}

private static String extractText(JsonObject choice, String fieldName) {
    if (choice == null) {
        return null;
    }
    if (choice.has("delta") && choice.get("delta").isJsonObject()) {
        JsonObject delta = choice.getAsJsonObject("delta");
        if (delta.has(fieldName)) {
            JsonElement value = delta.get(fieldName);
            if (value != null && !value.isJsonNull()) {
                return value.getAsString();
            }
        }
    }
    if (choice.has("message") && choice.get("message").isJsonObject()) {
        JsonObject message = choice.getAsJsonObject("message");
        if (message.has(fieldName)) {
            JsonElement value = message.get(fieldName);
            if (value != null && !value.isJsonNull()) {
                return value.getAsString();
            }
        }
    }
    return null;
}

record ParsedEvent(String content, String reasoning, boolean completed) {

    static ParsedEvent empty() {
        return new ParsedEvent(null, null, false);
    }

    static ParsedEvent done() {
        return new ParsedEvent(null, null, true);
    }

    boolean hasContent() {
        return content != null && !content.isEmpty();
    }

    boolean hasReasoning() {
```

```
        return reasoning != null && !reasoning.isEmpty();
    }
}
}
```

parseLine 接收一行 SSE 文本，返回一个 ParsedEvent。解析流程分五步：

- 1. 空行过滤：null 或空白行直接返回 ParsedEvent.empty()
- 2. 剥离data:前缀：SSE 每行数据都以 data: 开头，去掉前缀拿到真正的 payload
- 3. 识别[DONE]标记：OpenAI 协议约定的流结束标记，返回 ParsedEvent.done()
- 4. 解析JSON：把 payload 解析为 JsonObject，提取 choices[0]
- 5. 提取字段：从 choice0 中提取 content（增量内容）和 reasoning_content（思考内容，如果启用），检查 finish_reason 判断是否完成

整个类是 private 构造的工具类 (@NoArgsConstructor(access = PRIVATE))，所有方法都是 static，不维护状态——每行独立解析，无上下文依赖。

3. SSE 响应流走查：普通对话

用一个实际的 SSE 响应流来走查解析过程。用户问“你好”，百炼返回以下 SSE 数据 (reasoningEnabled = false)：

```
data: {"choices":[{"delta":{"role":"assistant","content":"","index":0}}]}

data: {"choices":[{"delta":{"content":"你","index":0}}]}

data: {"choices":[{"delta":{"content":"好","index":0}}]}

data: {"choices":[{"delta":{"content":"! "},"index":0}}]}

data: {"choices":[{"delta":{},"index":0,"finish_reason":"stop"}]}

data: [DONE]
```

逐行通过 parseLine 解析：

行号	SSE 行内容	payload (去前缀后)	ParsedEvent	doStream 中的回调动作
1	data: {"choices": [{"delta": {"role": "assistant", "content": ""}, ...}]}	JSON	("", null, false)	无动作 (hasContent() 为 false, 空字符串不推送)
2	(空行)	—	empty()	跳过 (doStream 中 line.isBlank() → continue)
3	data: {"choices": [{"delta": {"content": "你"}, ...}]}	JSON	("你", null, false)	callback.onContent("你")
4	(空行)	—	empty()	跳过
5	data: {"choices": [{"delta": {"content": "好"}, ...}]}	JSON	("好", null, false)	callback.onContent("好")
6	(空行)	—	empty()	跳过
7	data: {"choices": [{"delta": {"content": "! "}, ...}]}	JSON	("! ", null, false)	callback.onContent("! ")
8	(空行)	—	empty()	跳过

9	data: {"choices": [{"delta": {"reasoning_content": "用户在问候"}, "index": 0}]}	JSON	(null, null, true)	callback.onComplete() → completed = true → break
10	data: [DONE]	[DONE]	done()	不会被处理到（第 9 行已经 break 了）

注意第 1 行——第一个 chunk 通常只包含 role: "assistant" 和一个空的 content: ""。ParsedEvent 的 hasContent() 判断 content != null && !content.isEmpty(), 空字符串返回 false, 所以不会触发 callback.onContent()。这是合理的——空字符串推给前端没有意义。流结束有两个标记: 第 9 行的 finish_reason: "stop" 和第 10 行的 [DONE]。大多数供应商两个都会发。代码中两个都做了处理, 但 finish_reason 先到就已经触发 onComplete() 和 break 了, [DONE] 实际不会被读取到。这种双重检查是防御性设计——有些供应商只发其中一个。

4. SSE 响应流走查：深度思考

当 reasoningEnabled = true 时, 解析器会额外提取 reasoning_content 字段。看一个深度思考场景的 SSE 响应流：

```
data: {"choices":[{"delta":{"reasoning_content":"用户在问候"},"index":0}]}

data: {"choices":[{"delta":{"reasoning_content":"，我应该友好回应"},"index":0}]}

data: {"choices":[{"delta":{"content":"你好"},"index":0}]}

data: {"choices":[{"delta":{"content":"! "},"index":0}]}

data: {"choices":[{"delta":{},"index":0,"finish_reason":"stop"}]}

data: [DONE]
```

解析和回调过程：

SSE payload 关键字段	ParsedEvent	回调动作
delta.reasoning_content: "用户在问候"	(null, "用户在问候", false)	callback.onThinking("用户在问候")
delta.reasoning_content: "，我应该友好回应"	(null, "，我应该友好回应", false)	callback.onThinking("，我应该友好回应")
delta.content: "你好"	("你好", null, false)	callback.onContent("你好")
delta.content: "! "	("! ", null, false)	callback.onContent("! ")
finish_reason: "stop"	(null, null, true)	callback.onComplete()

先推送思考过程 (onThinking), 再推送正式回答 (onContent), 最后结束 (onComplete)。前端可以把思考过程显示在一个折叠区域里, 正式回答显示在主区域——两种内容通过不同的回调方法自然分流。如果 reasoningEnabled = false, 即使 JSON 中包含 reasoning_content 字段, parseLine 也会跳过提取——reasoning 直接赋值 null, hasReasoning() 返回 false, callback.onThinking() 不会被调用。这个开关由 AbstractOpenAIStyleChatClient 的钩子方法 isReasoningEnabledForStream(request) 控制, 默认根据 request.getThinking() 决定。

4.1 extractText 的双路径提取

```
private static String extractText(JsonObject choice, String fieldName) {
    if (choice == null) {
        return null;
    }
    // 路径一：从 delta 中提取（流式标准路径）
    if (choice.has("delta") && choice.get("delta").isJsonObject()) {
        JsonObject delta = choice.getAsJsonObject("delta");
        if (delta.has(fieldName)) {
            JsonElement value = delta.get(fieldName);
            if (value != null && !value.isJsonNull()) {
                return value.getAsString();
            }
        }
    }
}
```

```

    }
    // 路径二：从 message 中提取（兼容非标准行为）
    if (choice.has("message") && choice.get("message").isObject()) {
        JsonObject message = choice.getAsJsonObject("message");
        if (message.has(fieldName)) {
            JsonElement value = message.get(fieldName);
            if (value != null && !value.isJsonNull()) {
                return value.getAsString();
            }
        }
    }
    return null;
}

```

先尝试从 delta 中提取，再尝试从 message 中提取。流式响应的标准路径是 `choices[0].delta.content`，但某些供应商（或者某些模型在特定情况下）可能用 message 包装增量内容。双路径提取是一种防御性编程——不管供应商用 delta 还是 message，都能正确解析。优先检查 delta 是因为它是 OpenAI 流式协议的标准约定。

4.2 ParsedEvent record

```

record ParsedEvent(String content, String reasoning, boolean completed) {

    static ParsedEvent empty() {
        return new ParsedEvent(null, null, false);
    }

    static ParsedEvent done() {
        return new ParsedEvent(null, null, true);
    }

    boolean hasContent() {
        return content != null && !content.isEmpty();
    }

    boolean hasReasoning() {
        return reasoning != null && !reasoning.isEmpty();
    }
}

```

ParsedEvent 是解析结果的载体，用 Java 17 的 record 定义——不可变，自动生成 equals / hashCode / toString，适合做纯数据传递。两个工厂方法让语义更清晰：ParsedEvent.empty() 表示这一行没有有效内容（空行、无 choices 等），ParsedEvent.done() 表示流结束。调用方通过 hasContent() / hasReasoning() / completed() 判断该做什么，不需要手动检查 null。

StreamAsyncExecutor：异步提交

1. 完整代码

```

@NoArgsConstructor(access = lombok.AccessLevel.PRIVATE)
public final class StreamAsyncExecutor {

    private static final String STREAM_BUSY_MESSAGE = "流式线程池繁忙";

    static StreamCancellationHandle submit(Executor executor,
                                           Call call,
                                           StreamCallback callback,
                                           Consumer<AtomicBoolean> streamTask) {
        AtomicBoolean cancelled = new AtomicBoolean(false);
        try {
            CompletableFuture.runAsync(() -> streamTask.accept(cancelled), executor);
        } catch (RejectedExecutionException ex) {
            call.cancel();
        }
    }
}

```

```
        callback.onError(new ModelClientException(
            STREAM_BUSY_MESSAGE, ModelClientErrorType.SERVER_ERROR, null, ex));
        return StreamCancellationHandles.noop();
    }
    return StreamCancellationHandles.fromOkHttp(call, cancelled);
}
}
```

StreamAsyncExecutor 是一个无状态的工具类，只有一个 submit 方法，职责单一：把阻塞式的流式读取任务提交到专用线程池异步执行，并构建取消句柄。

2. submit 方法逐段讲解

四个参数的含义：

- executor——专用流式线程池（modelStreamExecutor），由 Spring 配置注入到 AbstractOpenAIStyleChatClient 中
- call——OkHttp 的 Call 对象，已经通过 httpClient.newCall(streamRequest) 创建但还没执行。用于构建取消句柄，以及线程池拒绝时的清理
- callback——流式回调，线程池拒绝时用于通知错误
- streamTask——实际的流式读取逻辑（就是 doStream 方法），类型是 Consumer<AtomicBoolean>，接收一个 cancelled 信号作为输入

执行流程：

- 第一步**，创建取消信号：AtomicBoolean cancelled = new AtomicBoolean(false)。这个 cancelled 是连接 doStream 读取循环和 StreamCancellationHandle.cancel() 的桥梁——取消句柄调 cancelled.set(true)，读取循环 while (!cancelled.get()) 检查到后退出。两个操作分别在不同线程上发生，AtomicBoolean 保证可见性。
- 第二步**，提交异步任务：CompletableFuture.runAsync(() -> streamTask.accept(cancelled), executor)。把流式读取逻辑提交到 executor 线程池上异步执行。streamTask.accept(cancelled) 就是调用 doStream(call, callback, cancelled, reasoningEnabled)。
- 第三步**，构建取消句柄并返回：StreamCancellationHandles.fromOkHttp(call, cancelled)。把 Call 和 cancelled 打包成一个取消句柄返回给调用方。调用方拿到句柄后可以随时调 cancel() 中断流式传输。

2.1 线程池拒绝的降级处理

如果线程池满了（所有线程都在处理其他流式请求），CompletableFuture.runAsync 会抛出 RejectedExecutionException。这时的降级逻辑：

- 1. call.cancel()——取消 OkHttp 请求，释放底层连接。虽然 call.execute() 还没被调用（任务没提交成功），但 cancel 一个未执行的 Call 是安全的，确保不会有悬挂的连接
- 2. callback.onError(new ModelClientException(...))——通过回调通知调用方线程池繁忙。注意这里不是抛异常，而是调 callback.onError()——因为 doStreamChat 的返回类型是 StreamCancellationHandle，回调模式下错误通过 onError 传递，不通过异常
- 3. return StreamCancellationHandles.noop()——返回一个空操作的取消句柄。因为流式任务实际没有启动，没有什么可取消的，noop 句柄的 cancel() 什么都不做

StreamCancellationHandle：取消机制

1. 接口定义

```
public interface StreamCancellationHandle {

    /**
     * 取消当前流式推理任务
     * 调用后应立即尝试取消底层模型生成过程
     */
    void cancel();
}
```

只有一个方法 cancel()，语义明确：取消当前正在进行的流式推理。典型使用场景是用户在前端点击停止生成——前端发一个取消请求到后端，后端调用 handle.cancel()，正在进行的流式读取被中断，不再继续推送内容。
cancel() 必须幂等——用户可能快速点击多次，不应该抛异常。

2. StreamCancellationHandles 工厂


```

@NoArgsConstructor(access = lombok.AccessLevel.PRIVATE)
public final class StreamCancellationHandles {

    private static final StreamCancellationHandle NOOP = () -> {};

    public static StreamCancellationHandle noop() {
        return NOOP;
    }

    public static StreamCancellationHandle fromOkHttp(Call call, AtomicBoolean cancelled) {
        return new OkHttpCancellationHandle(call, cancelled);
    }

    private static final class OkHttpCancellationHandle implements StreamCancellationHandle {

        private final Call call;
        private final AtomicBoolean cancelled;
        private final AtomicBoolean once = new AtomicBoolean(false);

        private OkHttpCancellationHandle(Call call, AtomicBoolean cancelled) {
            this.call = call;
            this.cancelled = cancelled;
        }

        @Override
        public void cancel() {
            if (!once.compareAndSet(false, true)) {
                return;
            }
            if (cancelled != null) {
                cancelled.set(true);
            }
            if (call != null) {
                call.cancel();
            }
        }
    }
}

```

两个工厂方法：

`noop()`——返回单例空操作句柄，`cancel()` 是一个空 `lambda () -> {}`。用于线程池拒绝等异常场景，流式任务根本没启动，没什么可取消的

`fromOkHttp(Call, AtomicBoolean)`——构建 `OkHttp` 取消句柄，这是正常流程使用的

3. OkHttpCancellationHandle 的双重取消

`OkHttpCancellationHandle.cancel()` 的执行逻辑：

```

public void cancel() {
    // ① CAS 保证只执行一次
    if (!once.compareAndSet(false, true)) {
        return;
    }
    // ② 设置取消信号
    if (cancelled != null) {
        cancelled.set(true);
    }
    // ③ 取消 OkHttp 连接
    if (call != null) {
        call.cancel();
    }
}

```

```
    }  
}
```

三步操作各有其职：

`once.compareAndSet(false,true)` ——CAS 保证取消逻辑只执行一次。如果 `cancel()` 被调用两次，第二次 `compareAndSet` 失败，直接 `return`。这就是幂等保证。

`cancelled.set(true)` ——设置取消信号。`doStream` 的读取循环每读完一行就检查 `while (!cancelled.get())`，下一次循环迭代时发现 `cancelled == true`，主动退出。这是协作式取消——读取循环自己检查信号并退出，而不是被强制中断。

`call.cancel()` ——取消 `OkHttp` 的 `HTTP` 连接。这一步直接关闭底层 `socket`，中断网络 `I/O`。

为什么需要双重取消？只设 `cancelled` 标志够不够？

——不够。如果 `doStream` 正好阻塞在 `source.readUtf8Line()` 上等待数据，即使 `cancelled` 被设为 `true`，循环也不会立即退出。因为循环条件 `!cancelled.get()` 是在每次迭代开头检查的，而 `readUtf8Line()` 正在阻塞等待网络数据，下一次循环条件检查要等到 `readUtf8Line()` 返回之后。如果供应商推送很慢（比如大模型正在思考一个复杂问题），这个阻塞可能持续很久。

`call.cancel()` 解决这个问题——它直接关闭底层 `socket`，让正在阻塞的 `readUtf8Line()` 立即抛出 `IOException`，从而快速退出。

两者协同工作：

`cancelled` 负责**循环正常退出路径** ——如果 `readUtf8Line()` 刚好返回了一行数据，下一次循环条件检查时退出

`call.cancel()` 负责**阻塞中断路径** ——如果 `readUtf8Line()` 正在阻塞等待数据，直接中断 `I/O`

3.1 取消后的异常处理

`call.cancel()` 导致 `readUtf8Line()` 抛出的 `IOException` 会被 `doStream` 最外层的 `catch (Exception e)` 捕获。但代码在 `catch` 块里检查了 `cancelled` 标志——如果是取消导致的异常，只打一条 `info` 日志（流式响应取消期间产生异常（可忽略）），不调 `callback.onError(e)`。这样调用方完全不需要区分取消异常和真实异常：收到 `onError` 回调就一定是真正的错误，取消场景下不会收到任何干扰。

doStream：逐行读取与回调推送

1. 完整代码

```
private void doStream(Call call, StreamCallback callback,  
    AtomicBoolean cancelled, boolean reasoningEnabled) {  
    try (Response response = call.execute()) {  
        if (!response.isSuccessful()) {  
            String body = HttpResponseHelper.readBody(response.body());  
            throw new ModelClientException(  
                provider() + " 流式请求失败: HTTP " + response.code() + " - " + body,  
                ModelClientErrorType.fromHttpStatus(response.code()),  
                response.code()  
            );  
        }  
        ResponseBody body = response.body();  
        if (body == null) {  
            throw new ModelClientException(  
                provider() + " 流式响应为空",  
                ModelClientErrorType.INVALID_RESPONSE, null);  
        }  
        BufferedSource source = body.source();  
        boolean completed = false;  
        while (!cancelled.get()) {  
            String line = source.readUtf8Line();  
            if (line == null) {  
                break;  
            }  
            if (line.isBlank()) {  
                continue;  
            }  
            try {  
                OpenAIStyleSseParser.ParsedEvent event =  
                    OpenAIStyleSseParser.parseLine(line, gson, reasoningEnabled);  
                if (event.hasReasoning()) {  
                    callback.onThinking(event.reasoning());  
                }  
                if (event.hasContent()) {
```

```
        callback.onContent(event.content());
    }
    if (event.completed()) {
        callback.onComplete();
        completed = true;
        break;
    }
} catch (Exception parseEx) {
    log.warn("{} 流式响应解析失败: line={}, provider(), line, parseEx);
}
}

if (cancelled.get()) {
    log.info("{} 流式响应已被取消", provider());
    return;
}

if (!completed) {
    throw new ModelClientException(
        provider() + " 流式响应异常结束",
        ModelClientErrorType.INVALID_RESPONSE, null);
}
} catch (Exception e) {
    if (!cancelled.get()) {
        callback.onError(e);
    } else {
        log.info("{} 流式响应取消期间产生异常 (可忽略): {}", provider(), e.getMessage());
    }
}
}
```

这个方法运行在 modelStreamExecutor 线程池的异步线程上，是整个流式调用的核心。

2. 执行流程逐段讲解

HTTP请求发送： call.execute() 发起 HTTP 连接，阻塞等待响应头到达。注意这一步是在异步线程上执行的，阻塞不影响 Tomcat 线程。try-with-resources 保证 Response 在方法退出时正确关闭。

响应状态码校验： 和 doChat 一样——非成功状态码读取错误响应体，按状态码分类错误类型，抛 ModelClientException。

获取BufferedSource： response.body().source() 拿到 OkHttp 的缓冲数据源，后续通过它逐行读取 SSE 数据。

读取循环： while (!cancelled.get()) 是循环条件——每轮迭代先检查取消信号，cancelled 为 true 时立即退出。
循环体内的处理：

- 1. source.readUtf8Line()——阻塞读取一行。返回 null 表示连接已关闭（流结束或异常断开）
- 2. line.isBlank() → continue——跳过空行。SSE 协议中空行是事件之间的分隔符
- 3. OpenAIStyleSseParser.parseLine(line, gson, reasoningEnabled)——解析 SSE 行
- 4. event.hasReasoning() → callback.onThinking(event.reasoning())——有思考内容就推送
- 5. event.hasContent() → callback.onContent(event.content())——有增量内容就推送
- 6. event.completed() → callback.onComplete() → break——流结束，通知完成并退出循环

3. 三种结束方式

读取循环退出后，有三种可能：

结束方式	触发条件	循环退出原因	后续处理
正常完成	finish_reason 非 null 或 [DONE]	event.completed() → break	callback.onComplete() 已在循环内调用，completed = true
用户取消	调用 handle.cancel()	cancelled.get() == true, 循环条件不满足	打印日志 流式响应已被取消，直接 return，不调任何回调
异常中断	连接意外断开	readUtf8Line() 返回 null → break	completed == false, 抛 ModelClientException(INVALID_RESPONSE) → callback.onError()

循环退出后的判断分两步：先检查 `cancelled`——如果是用户取消，打一条 `info` 日志就直接 `return`，不走后续的异常抛出逻辑；再检查 `completed`——如果不是取消也不是正常完成，说明流意外结束，抛 `ModelClientException` 通知调用方。

最外层的 `catch (Exception e)` 也区分了取消和非取消两种情况：

```
} catch (Exception e) {
    if (!cancelled.get()) {
        callback.onError(e);
    } else {
        log.info("{} 流式响应取消期间产生异常（可忽略）：{}", provider(), e.getMessage());
    }
}
```

非取消场景下的异常（HTTP 状态码错误、响应体为空、流异常中断）通过 `callback.onError(e)` 传递给调用方。取消场景下的异常（`call.cancel()` 导致的 `IOException`）只打日志不调 `onError`——这是合理的，用户主动取消是预期行为，不应该触发错误回调。这样调用方不需要自己区分取消异常和真实异常，`onError` 被调用就一定是真正的错误。

保证 `StreamCallback` 的四个方法中：正常完成走 `onComplete()`，异常走 `onError()`，用户取消不调任何终态方法（调用方通过自己触发了 `cancel()` 已经知道流结束了）。

4. 解析异常的容错处理

循环体内有一个 `try-catch` 包住了 `parseLine` 和后续的回调用：

```
try {
    OpenAIStyleSseParser.ParsedEvent event = OpenAIStyleSseParser.parseLine(line, gson, reasoningE
    // ... 回调
} catch (Exception parseEx) {
    log.warn("{} 流式响应解析失败：line={}", provider(), line, parseEx);
}
```

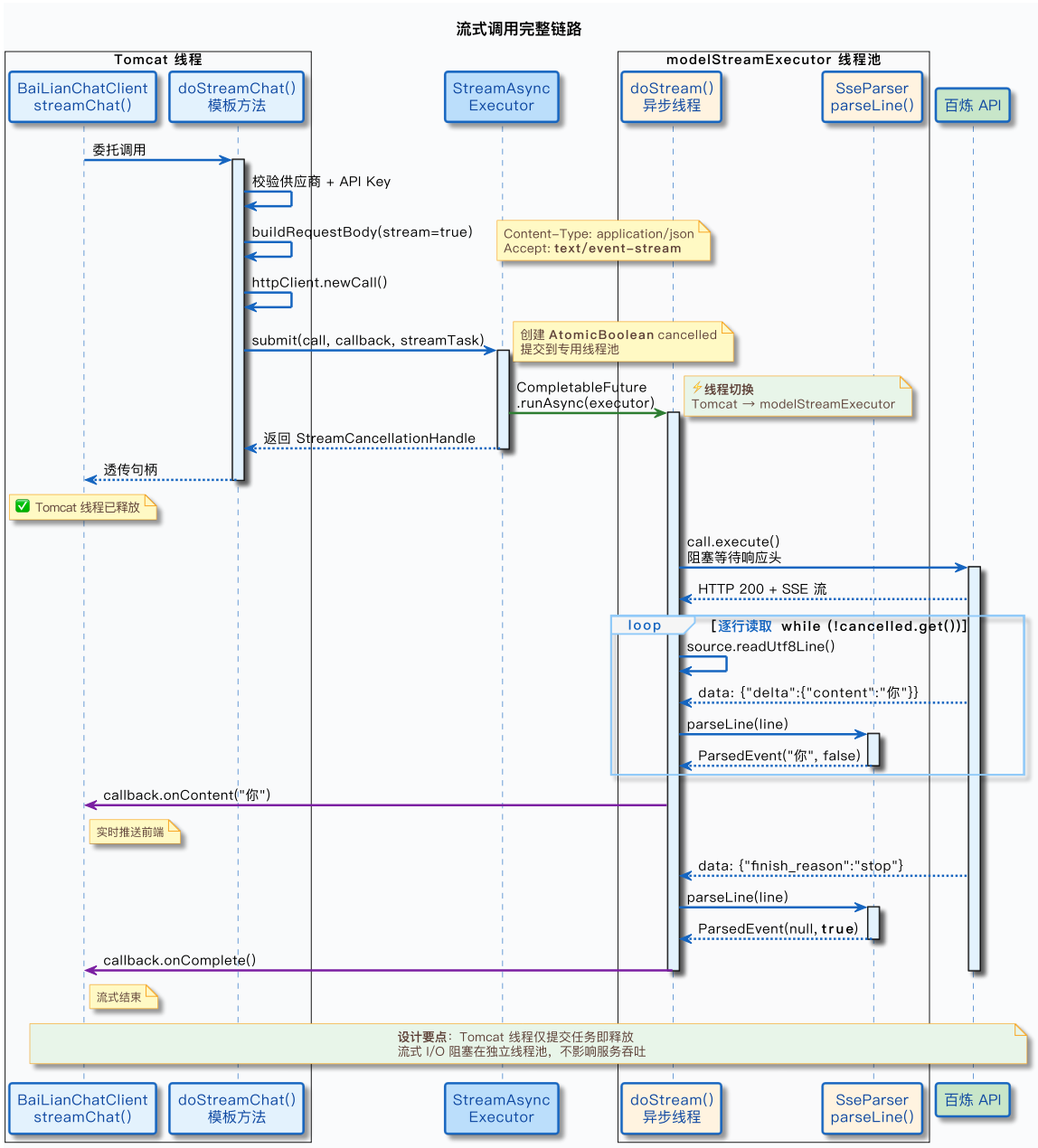
单行解析失败只打警告日志，不中断整个流。为什么？

某些供应商偶尔会发出非标准格式的行——注释行（`:heartbeat`）、格式错误的 JSON、意外的空对象等。如果一行解析失败就中断整个流，用户在对话过程中突然收到一个错误，体验很差。实际上关键数据（`content`、`finish_reason`）分散在多行里，跳过一行坏数据，下一行大概率是正常的。

这种容错策略在流式场景下很常见——宁可丢一个 `token`，也不要断整个流。

完整调用链路图

把所有组件串起来，看一次完整的流式调用链路：

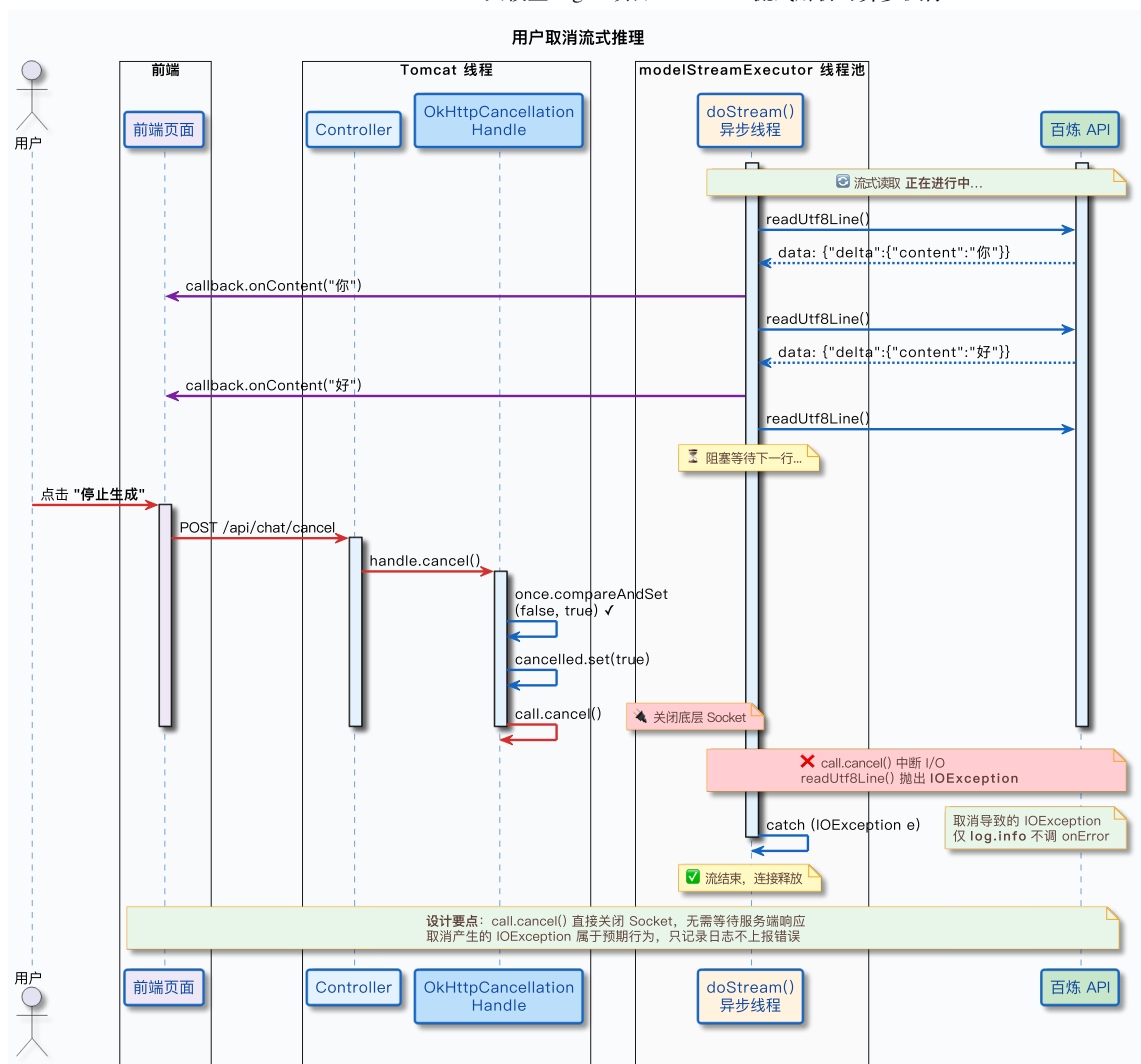


整条链路分三个阶段:

1. **请求构建阶段** (Tomcat 线程): 校验 → 构建请求体 → 创建 Call → 提交异步任务 → 返回取消句柄。这个阶段很快, Tomcat 线程立即释放
2. **连接建立阶段** (异步线程): call.execute() 发起 HTTP 连接, 阻塞等待供应商响应。这一步的耗时取决于网络延迟和供应商的处理速度
3. **流式读取阶段** (异步线程): 逐行读取 SSE 数据 → 解析 → 回调推送。这个阶段持续整个模型生成过程, 可能几秒到几分钟

取消场景走查

走一个完整的取消场景。假设用户问了一个复杂问题, 模型正在生成回答, 用户觉得不需要了, 点击停止生成。



走查细节:

1. 流式读取已经进行了一段时间, callback.onContent("你") 和 callback.onContent("好") 已经推送完成
2. 用户点击停止生成, 前端发取消请求到后端
3. 后端拿到之前保存的 StreamCancellationHandle, 调用 handle.cancel()
4. OkHttpCancellationHandle.cancel() 执行三步: once CAS 成功 → cancelled.set(true) → call.cancel()
5. 此时 doStream 有两种情况:
 - 情况A: 如果 readUtf8Line() 刚好返回了一行数据, 进入下一次循环。while (!cancelled.get()) 检查到 cancelled == true, 循环退出。循环后 cancelled.get() 为 true, 打印日志 流式响应已被取消, 直接 return
 - 情况B: 如果 readUtf8Line() 正阻塞等待数据 (如图所示), call.cancel() 关闭 socket → readUtf8Line() 抛出 IOException → 最外层 catch 捕获 → 检查 cancelled 为 true → 只打日志 流式响应取消期间产生异常 (可忽略), 不调 callback.onError()
6. 不管哪种情况, 取消都不会触发 callback.onError()——调用方通过自己触发了 cancel() 已经知道流结束了。try-with-resources 确保 Response 正确关闭, 连接资源释放

三个供应商客户端的 streamChat

和上一篇讲的 chat 方法一样, 三个供应商客户端的 streamChat 也是一行委托:

```

// OllamaChatClient
@Override
@RagTraceNode(name = "ollama-stream-chat", type = "LLM_PROVIDER")
public StreamCancellationHandle streamChat(
    ChatRequest request, StreamCallback callback, ModelTarget target) {
    return doStreamChat(request, callback, target);
}

```

```
// BaiLianChatClient
@Override
@RagTraceNode(name = "bailian-stream-chat", type = "LLM_PROVIDER")
public StreamCancellationHandle streamChat(
    ChatRequest request, StreamCallback callback, ModelTarget target) {
    return doStreamChat(request, callback, target);
}
```

```
// SiliconFlowChatClient
@Override
@RagTraceNode(name = "siliconflow-stream-chat", type = "LLM_PROVIDER")
public StreamCancellationHandle streamChat(
    ChatRequest request, StreamCallback callback, ModelTarget target) {
    return doStreamChat(request, callback, target);
}
```

三者完全一致——一行 `return doStreamChat(request, callback, target)`，加一个 `@RagTraceNode` 注解。供应商之间的差异（有没有 API Key、请求体有没有特殊字段、流式解析时是否提取 `reasoning_content`）全部通过基类的钩子方法处理了，子类不需要关心流式读取的任何细节。

小结与下一步

回顾这一篇的核心要点：

`doStreamChat` 和 `doChat` 前半部分相同（校验 + 构建请求），后半部分完全不同：同步调用在当前线程同步执行并返回 `String`，流式调用把 `Call` 交给 `StreamAsyncExecutor` 异步执行并返回取消句柄

`StreamCallback` 定义了四个回调方法（`onContent` / `onThinking` / `onComplete` / `onError`），通过回调模式实现增量推送。`onThinking` 的 `default` 空实现让不关心思考过程的调用方零成本忽略

`OpenAIStyleSseParser` 是一个针对 `OpenAI` 兼容协议优化的 SSE 行解析器，处理 `data`：前缀剥离、`[DONE]` 识别、`delta.content` / `delta.reasoning_content` 提取、`finish_reason` 判断。`extractText` 的双路径提取（先 `delta` 后 `message`）兼容非标准供应商行为

`StreamAsyncExecutor` 将阻塞式的流式读取提交到专用线程池异步执行，解放 `Tomcat` 请求线程。线程池拒绝时优雅降级：取消连接 + `callback.onError()` + 返回 `noop` 句柄

`OkHttpCancellationHandle` 实现双重取消——`AtomicBoolean cancelled` 信号通知读取循环退出 + `Call.cancel()` 中断阻塞 I/O——两者协同保证取消的及时性。`AtomicBoolean once` 保证 `cancel()` 幂等

`doStream` 中解析异常只打日志不中断流，容错处理保证单行坏数据不影响整体体验。外层 `catch` 区分取消与非取消——取消导致的异常只打日志不调 `onError`，调用方不需要自己过滤取消异常

流式调用有三种结束方式：正常完成（`finish_reason` 或 `[DONE]`）、用户取消（`cancelled == true`，打日志直接 `return`）、异常中断（流意外结束，`callback.onError()`）

不过，本篇讲的是供应商级别的流式调用——一个 `ChatClient` 如何连接一个供应商的 API 并流式读取响应。在路由层面，还有一个更大的问题没有解决：`RoutingLLMService.streamChat()` 怎么做流式路由？

回想之前讲的 `executeWithFallback`——同步调用可以逐个尝试候选，调供应商 A 失败了，换供应商 B 再调，成功了 `return` 结果。整个重试过程发生在 `return` 之前，调用方一直在阻塞等待，完全不知道中间换了几个供应商，最终拿到一个 `String` 就行。

流式调用就不一样了。`streamChat()` 调用后立即 `return` 一个取消句柄，真正的数据在异步线程里通过回调推送。问题在于：等异步线程发现供应商 A 不行（`onError` 被调用了），`doStreamChat` 方法早就 `return` 了——取消句柄已经交到调用方手里了。这时候你想切换到供应商 B 重试，`streamChat()` 会产生一个新的取消句柄，但调用方拿着的还是供应商 A 的旧句柄，新句柄没有办法交回去。

一句话概括：同步调用的 `fallback` 发生在 `return` 之前，可以换了再返回；流式调用的 `fallback` 需要发生在 `return` 之后，但句柄已经交出去了，没有第二次返回的机会。

下一篇进入流式路由的首包探测机制——`RoutingLLMService` 如何用 `ProbeStreamBridge` 探测桥接器（基于 `CompletableFuture` 信号 + `List<Runnable>` 缓冲）实现 `probe-and-commit` 模式：先用一个探测回调接收第一个 `token`，确认供应商可用后再把缓冲的事件 `commit` 给真实回调。如果首包探测失败，切换到下一个供应商重试——对业务层完全透明。